# Product Requirements Document (PRD) & Research Guide

## Explorations Search & Filter for a Learning Portal

*Final version, aligned with the OJT Project Design Template.*

## 1. Project Overview

| Attribute | Details |
|---|---|
| Project Name | Develop Search & Filter for Explorations for a Learning Portal |
| Assignee | Ananvaya Bhishmacharya Wadgure (ID: PST-26-0041, Group: G380) |
| Roll No / Year & Section | [Fill in before submitting] |
| Tags | PST_OSS_043, open_source |
| Stack / Framework | Angular; HTML, CSS, JavaScript; TypeScript; VS Code, Git, Angular CLI, Browser DevTools |
| Mentor | [Mentor name] |
| Version | Final |

## 2. Research Document: Core Concepts

### 2.1 Problem Statement

On an Oppia-style learning portal, students, educators and lesson creators browse a gallery of explorations (interactive lessons). As the gallery grows, finding a suitable lesson by scrolling becomes inefficient. This project builds an advanced gallery search with dynamic filters for subject, difficulty and language, so users can narrow results precisely.

### 2.2 Why It Matters

A growing library is useless if content cannot be discovered. Without robust search, learners leave before finding material at the right level or in their language, and educators struggle to find lessons for a specific subject. Better filtering improves discovery across languages and skill levels and gives the open-source community an improvement that benefits every user.

### 2.3 Version Control: Git and GitHub

- Git is a distributed version control system. It lets several developers work at once, create branches for new features (such as this search filter) and merge changes without overwriting each other's work.
- GitHub hosts Git repositories in the cloud. It provides issue tracking, pull request review and automation through GitHub Actions, which this project uses for its CI pipeline.

### 2.4 Target Repository and Packaging

The repository context and packaging approach are:

**Approach:** The feature is first built as a standalone Angular module (search service, filter components and results list) that follows Oppia's coding and contribution conventions.

**Why:** A standalone module is realistic for a 4-week timeline and can be demonstrated independently. Its structure is kept clean so it can later be adapted for an upstream contribution to Oppia.

**Confirmation:** The target repository and packaging format (standalone demo versus upstream Oppia module) will be confirmed with the mentor in Week 1, because it affects setup effort.

# 3. Functional Scope

## 3.1 Core Features (Must-Haves)

1. Gallery keyword search: case-insensitive matching of the search text against exploration titles and descriptions.
2. Dynamic multi-filters: Subject, Difficulty and Language filters that work alone or together with keyword search, combined with AND logic across filter types.
3. Results display and states: a results list that updates dynamically, a clear-all-filters control, and explicit empty-result and error states.
4. Unit and component testing with a GitHub Actions CI workflow: tests pass with 85% or higher coverage, and CI runs on every push and pull request. These are acceptance criteria, so they are must-haves.

## 3.2 Stretch Goals

- Sorting and pagination of large result sets.
- URL state management: selected search text and filters stored in the page URL for shareable links.
- Accessibility: keyboard navigation and screen reader labels for the filter controls.
- Upstream preparation: packaging the module for a possible contribution to Oppia.

# 4. Data Source and Metadata

## 4.1 Data Source

The first version uses a typed mock JSON dataset of about 40 explorations, accessed through a service interface. Because components depend only on the service, the mock source can later be replaced with Oppia's real exploration data without changing the UI. This also explains the "API calls / data binding" work in Week 2: the service exposes an asynchronous method that returns the exploration list, as a real API would.

## 4.2 Metadata Fields

Each exploration has the following fields:

| Field | Type | Notes |
|---|---|---|
| id | string | Unique identifier |
| title | string | Searchable |
| description | string | Searchable |
| subject | string | Filter, for example Mathematics, Science |
| difficulty | string | Filter: Beginner, Intermediate, Advanced |
| language | string | Filter, for example English, Hindi |

```
interface ExplorationMetadata {
```

```
  id: string;
  title: string;
  description: string;
  subject: string;
  difficulty: 'Beginner' | 'Intermediate' | 'Advanced';
  language: string;
}
```

Missing or malformed fields are handled gracefully (the record is skipped or treated as non-matching) and are covered by tests.

### 4.3 Definition of Error States

An error state is shown to the user in any of these cases:

1. The exploration data fails to load.
2. A record has missing or malformed metadata.
3. The page URL contains invalid filter values (stretch goal, URL state).

An empty-result state (no lesson matches) is different from an error state and has its own clear message with a clear-filters action.

## 5. System and Design Thinking

### 5.1 App Flow

User types search text and selects filters, then the Angular component or service matches against exploration metadata (AND logic), and the UI shows the filtered list, an empty-result message, or an error message.

### 5.2 Planned Interface Components

- Search box (with optional debouncing)
- Three filter dropdowns: Subject, Difficulty, Language
- Clear-all-filters button
- Results list showing title, description and metadata tags
- Empty-result and error message areas

### 5.3 Data Structures and Algorithms

- Data structures: TypeScript interfaces for metadata; arrays of exploration objects; Sets or Maps to track active filter values.
- Algorithms: case-insensitive text matching; combined filtering (AND across filter types); optional sorting and pagination; optional debouncing of search input.

### 5.4 Non-Functional Requirements

- Results update without a noticeable delay on the sample dataset (target: under 200 ms after input settles).
- Works in current versions of Chrome, Firefox and Edge.
- Layout is responsive for desktop and mobile widths.

## 6. Testing Plan

Unit tests check text matching and filter combinations using sample data with known expected results. Component tests check that typing and selecting filters update the list and that clearing

filters restores it. Manual checks are done in the browser, and coverage is measured against the 85% target in CI.

| # | Input | Expected result |
|---|---|---|
| 1 | Empty search, no filters | All explorations shown |
| 2 | Search "fraction" (any letter case) | Only items with the word in title or description |
| 3 | Subject = Mathematics | Only Mathematics items |
| 4 | Subject + Difficulty + Language together | Only items matching all three (AND) |
| 5 | Search text plus filters | Items matching text and all filters |
| 6 | Search text with no matches | Empty-result message shown |
| 7 | Special characters (for example %, (, \) | No crash; treated as plain text |
| 8 | Filters that cancel each other out | Empty-result message shown |
| 9 | Record with missing metadata | Handled without crashing; covered by a test |
| 10 | Click Clear filters | Full list restored |
| 11 | Data fails to load | Error message shown |

## 7. Timeline and Milestones (4 Weeks)

CI is set up in Week 1, and stretch work is spread across Weeks 3 and 4 so the final week is not overloaded.

| Phase | Focus | Deliverables and Tasks | Mentor Checkpoint |
|---|---|---|---|
| Week 1 | Foundation and setup | Dev environment for Oppia-style Angular work; repository scaffolding; confirm target repo with mentor; define the metadata interface; create the mock dataset; set up the GitHub Actions CI workflow | [ ] |
| Week 2 | Core search build | Gallery keyword search; basic forms; data service and data binding; first unit tests | [ ] |
| Week 3 | Filtering and refinement | Subject, difficulty and language filters; clear-filters control; empty and error states; validation; component tests; start accessibility and URL state | [ ] |
| Week 4 | Finalization and delivery | End-to-end testing; sorting and pagination if time permits; finish URL state and accessibility; reach 85% coverage; README with setup, architecture and contribution guidelines; 3-minute video walkthrough | [ ] |

## 8. Risks and Dependencies

**Technical risks:** learning Angular's component structure; setting up and navigating a large open-source repository; understanding how difficulty and language metadata is represented; combining several filters without confusing behavior; meeting the 85% coverage target.

**Help needed from mentors:** guidance on the Oppia gallery code and conventions; how exploration metadata is structured; whether to package the work as a module or standalone demo; code review; advice on contribution conventions.

## 9. Evaluation and Acceptance Criteria

- Unit and component testing: all tests pass with 85% or higher code coverage.
- CI/CD integration: the GitHub Actions workflow runs successfully on all pushes and pull requests.
- Functional accuracy: search and combined filters return correct results against the sample dataset.
- Resilience: empty and error states are handled with clear user feedback.
- Documentation: the README documents setup, architectural decisions and usage.

### Proof the project works

- GitHub repository link, README with screenshots of search and filters, and a 3-minute demo video.
- Unit and component test results, a coverage report, GitHub Actions runs and the test case table above.
- Feedback from the mentor and from peers acting as learners or educators, collected with a short question: "Was it easier to find a suitable lesson?"

## 10. Responsibilities

| **Task** | **Ananvaya Bhishmacharya Wadgure** | **Mentor** | **Mentor Notes** |
|---|---|---|---|
| Mock data model and search logic | [x] | [ ] | Review data model and approve approach |
| Filter controls and results UI | [x] | [ ] | Review UI and usability |
| Sorting, pagination, URL state, accessibility (stretch) | [x] | [ ] | Advise on priorities if time is short |
| Testing, CI workflow, README and demo video | [x] | [ ] | Review tests, CI results and documentation |

## 11. Sign-off

| | |
|---|---|
| Student signature(s) | _____ |
| Mentor approval | _____ |
| Date | _____ |